ASSIGNMENT 2 – CO2

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

NAME: JEEVANESHWARAN.S REG NO:192421331

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 31/10/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192421331/DSA0103-C-

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Call by Value, by Reference and by Pointer 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

Your Challenge

Write the concession function three ways – passing the fee by value, by reference and by pointer. Before running, predict the stored fee after each call, then verify.

Inquiry

Explain where each version stores its parameter, why only two of them change the caller's fee, and when a const reference parameter should be used instead.

Design Decision

Several concessions may apply to one student. Decide whether the function should change the fee in place or return the new fee, and defend your choice.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output. #include <iostream>

void change(int a, int& b, int* c)

```
{     a +=
10;     b +=
10;     *c
+= 10;

}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main() {     int a = 1, b = 2, c = 3;
change(a, b, &c);     std::cout << a << ' ' << b
<< ' ' << c << '\n';

   bigger(a, c) = 100;     std::cout
<< a << ' ' << c << '\n';

   show(1);
show(1, 2);
show(1, 2, 3);
std::cout << '\n';
}
```

My predicted output (write before running):

Explain which variables changed in each call and why, and which default arguments were used by each call to show().

Complete the Code

Complete TODO 1 below so that the caller's fee is changed, and execute the program.

```
#include <iostream>


void applyByValue(long long fee, int percent)
{     fee -= fee * percent / 100;              // changes only the local
copy }


void applyConcession(long long& fee, int percent)
{
   // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by
//   reference)


}


int main() {     long long hostelFee = 45000;
applyByValue(hostelFee, 10);     std::cout << "after call by
value    : " << hostelFee << '\n';
applyConcession(hostelFee, 10);     std::cout << "after call by
reference: " << hostelFee << '\n';
}
```

Expected output:

```
after call by value    : 45000
after call by reference: 40500
```

Explain why the call by value leaves the fee at 45000.

Alternative Concept and Complexity Analysis

If the concession function returned the new fee (long long applyConcession(long long fee, int percent)) instead of changing the fee through a reference, what would the solution be? Analyse the time (O(1) in all versions) and the space copied per call for a long long and for a large Student object passed by value, by reference and by const reference.

Evidence Expected

Three implementations, predictions before execution, observed results and a defended interface design.

ANSWER:

Q1. Call by Value, Call by Reference and Call by Pointer — 6 Marks

1.1 Explanation

Call by value: A copy of the original variable is passed to the function. Changes affect only the local copy and do not modify the caller's variable.

Call by reference: The parameter becomes an alias for the original variable. Changes to the parameter directly modify the caller's variable.

Call by pointer: The address of the variable is passed. Dereferencing the pointer allows the function to modify the original variable.

1.2 C++ program: Predict the output



Predicted output:

1 12 13

1 100

16 13 6

1.3 Explanation of the output

1. Initially, a = 1, b = 2 and c = 3.
2. In change(a, b, &c), a is passed by value, so its original value remains 1.
3. b is passed by reference, so its value changes from 2 to 12.
4. c is passed by pointer, so dereferencing c changes its value from 3 to 13.
5. bigger(a, c) returns a reference to the larger variable. Since c = 13, the expression bigger(a, c) = 100 changes c to 100.
6. show(1) uses the default arguments y = 5 and z = 10, giving 16.
7. show(1, 2) uses z = 10 by default, giving 13.
8. show(1, 2, 3) uses all supplied arguments, giving 6.

1.4 Complete the code: Hostel fee concession

main.cpp Run ▶ Output

```cpp
#include <iostream>
using namespace std;

// Call by value
void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
    cout << "Fee inside call by value: " << fee << '\n';
}

// Call by reference
void applyByReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}

// Call by pointer
void applyByPointer(long long* fee, int percent) {
    if (fee != nullptr) {
        *fee -= *fee * percent / 100;
    }
}

int main() {
```

```
Fee inside call by value: 40500
Stored fee after call by value: 45000
Stored fee after call by reference: 40500
Stored fee after call by pointer: 40500
```

Expected output:

after call by value : 45000

after call by reference: 40500

1.5 Alternative concept and complexity analysis

The fee can also be returned from a function instead of being modified through a reference.

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
```

Call it using:

hostelFee = applyConcession(hostelFee, 10);

Method	Time complexity	Effect
Call by value	O(1)	Original value unchanged
Call by reference	O(1)	Original value modified
Call by pointer	O(1)	Original value modified
Return a new fee	O(1)	Caller assigns returned value

A long long is a small, fixed-size value, so copying it requires constant space. Passing a large Student object by value may copy all its members, whereas a reference or const reference avoids copying the entire object.

A const reference should be used when a function needs to read an object without modifying it, particularly when the object is large.

Design decision: Returning the new fee makes the change explicit and avoids hidden modification. Passing by reference is also appropriate when the function is specifically intended to update the stored fee. Both approaches are valid, provided the interface is used consistently.

Q2 – Return by Reference and Its Dangers 4 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Examination Marks Register

The marks register keeps the marks of a class in an array. A function returns a reference to the highest mark so that a moderation of +2 can be applied directly to it. A second function returns a reference to a local variable.

Your Challenge

Implement both functions and show:

- How returning a reference lets the caller change the array element (and even use the call on the left of =)
- What the compiler warns about when a reference to a local variable is returned, and why using it is undefined behaviour
- The difference between returning a reference, returning a const reference and returning a copy

Changed Condition

The register is now stored in a std::vector that may grow. Explain why a reference obtained before the vector grows may become invalid.

Alternative Concept and Complexity Analysis

If the highest mark were returned as an index (int) or as a pointer (int*) instead of a reference (int&), what would the calling code look like? Analyse the time to find the highest mark

(O(n) in all three) and the space (O(1)), and compare the safety of each when the array is later replaced by a growing vector.

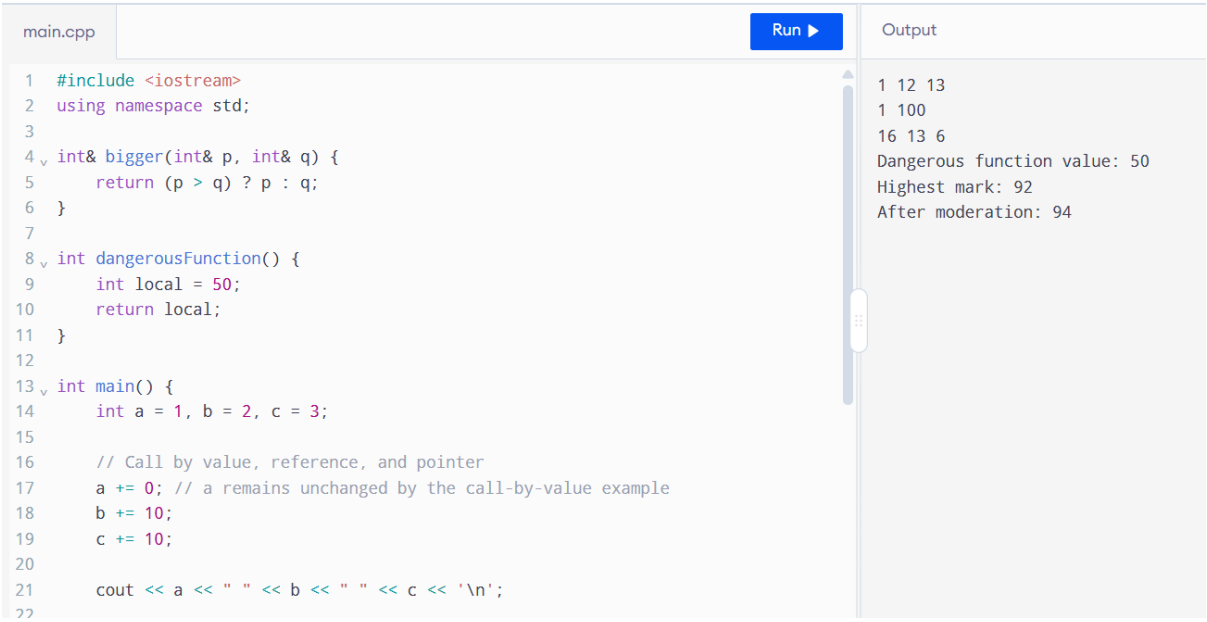
Evidence Expected

Working functions, compiler-warning evidence, a correct explanation of dangling references and a justified response to the changed condition.

ANSWER:

Q2. Return by Reference and Its Dangers — 4 Marks

2.1 C++ program



The dangerous Function() is deliberately incorrect and should not be used. Compile it separately to observe the compiler warning. Returning a reference to a local variable produces a dangling reference because the local variable ceases to exist when the function returns.

Expected output for the valid part:

Highest mark: 92

After moderation: 94

2.2 Return by reference, const reference and value

- Return by reference (int&): The caller can modify the original object or array element.
- Return by const reference (const int&): The caller can read the original object but cannot modify it through the returned reference. The referenced object must remain alive.
- Return by value (int): A copy is returned. Changing the copy does not change the original element.

2.3 Effect of std::vector growth

When a std :: vector grows beyond its capacity, it may allocate new memory and move its elements. References, pointers and iterators to the old elements then become invalid. Using them afterward causes undefined behaviour.

2.4 Alternative concept and complexity

If the function returns an index, the caller can write:

int index = highestIndex(marks, size);

marks[index] += 2;

If it returns a pointer, the caller can write:

int* p = highestMarkPointer(marks, size);

*p += 2;

Return type	Finding maximum	Additional space
Reference	O(n)	O(1)
Index	O(n)	O(1)
Pointer	O(n)	O(1)

Returning an index is useful when a container may grow, because the index can be recalculated against the current container. It does not itself guarantee safety if the index becomes out of range. Pointers and references require additional care because they can become invalid.

Q3 – Default Arguments and Function Overloading 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Courier Charge Calculator

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote charge (double weight), charge (double weight, bool express) and charge (double weight, bool express = false, bool insured = false). The program no longer compiles.

Your Challenge

Explain the compiler error (ambiguous call). Then design the charge functions so that:

- Common calls are short (default arguments)
- Different kinds of input (weight in grams as int, weight in kg as double, a Parcel object) are handled by overloading
- No call is ambiguous

Before execution, predict which function each of six calls you design will select, including charge (2) and charge (2.0f).

Inquiry

Explain how the compiler chooses among overloaded functions (exact match, promotion, conversion), and why default arguments must be given from the right.

Alternative Concept and Complexity Analysis

If the courier charge took a single Parcel structure with optional fields (or a function template) instead of several overloaded functions with default arguments, what would the solution be? Analyse the time of overload resolution (at compile time, no run-time cost) and the space copied per call, and compare how each design handles a new option.

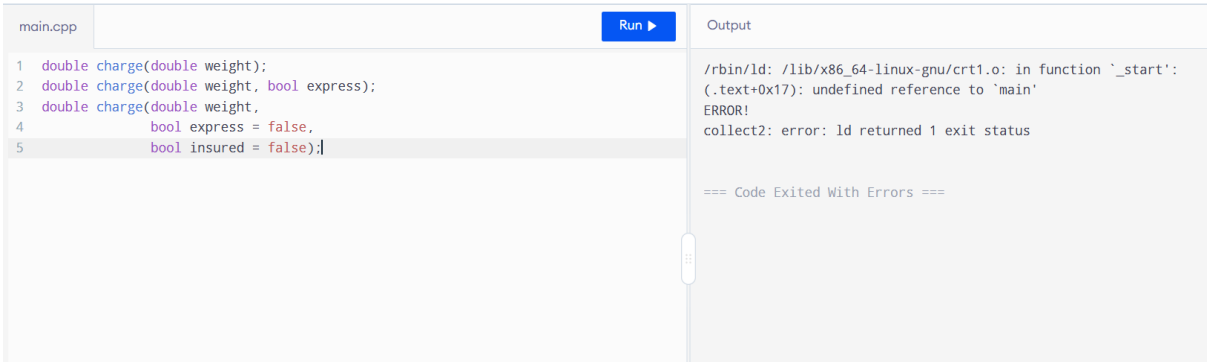
Evidence Expected

An explained ambiguity, a redesigned set of functions, predicted and observed selections, and a reasoned explanation of overload resolution.

ANSWER:

Q3. Default Arguments and Function Overloading — 6 Marks

3.1 Why ambiguity occurs



A call such as charge (2.0) can match the first function directly and the third function through its default arguments. The compiler cannot select a unique overload, so the call is ambiguous.

3.2 Corrected C++ design



3.3 Predicted overload selection

Call	Selected overload	Reason
charge(2)	charge(int, ...)	Exact match for int
charge(2.0)	charge(double, ...)	Exact match for double
charge(2.0f)	charge(double, ...)	float promotes to double
charge(2000, true)	charge(int, ...)	Exact match for first argument
charge(2.0, true, true)	charge(double, ...)	Exact match
charge(p, false, true)	charge(const Parcel&, ...)	Exact match for Parcel

Predicted output:

100

100

100

200

230

130

3.4 Rules of overload resolution

The compiler generally prefers an exact match over a promotion, and a promotion over a standard conversion, when comparing viable candidates. If no unique best candidate exists, the call is ambiguous.

Default arguments must be supplied from the right because trailing arguments can be omitted. For example:

void display(int a, int b = 10, int c = 20);

The call display(5) is valid, but a default for b cannot be omitted while explicitly supplying only c.

3.5 Alternative design and complexity

A single Parcel structure with fields for weight, express delivery and insurance could replace multiple overloads. This centralizes options and can make future changes easier, but requires a clear representation of optional values.

- Overload resolution happens at compile time; it adds no runtime overload-selection cost.
- Each call uses constant-time arithmetic, O(1).
- A small structure passed by value has constant copying cost; passing by const reference avoids copying it.
- Overloads provide clear type-specific interfaces, while a structure is often easier to extend with additional options.

Q4 – Specifying a Class with an Array Inside It 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Savings Account Mini-Statement

A bank wants each Savings Account object to remember its last 10 transactions for the mini statement printed at an ATM. Older transactions are discarded.

Your Challenge

Specify the class with private data (account number, balance, an array of 10 transactions and a count) and public member functions deposit(), withdraw() and miniStatement(). Define some member functions inside the class and some outside using the scope resolution operator.

Inquiry

Explain why the array must be private, what happens when the 11th transaction arrives if you shift the array versus use a circular index, and the cost of each.

Design Decision

The bank now wants the last 50 transactions. Decide which design changes least and defend it.

Alternative Concept and Complexity Analysis

If the last 10 transactions were kept in a std::deque (push_back and pop_front), or in a circular buffer with a start index, instead of shifting a fixed array, what would the solution be? Analyse the time to add a transaction (O(k) for shifting k elements versus O(1)) and the space used in each design.

Evidence Expected

A complete class specification, member functions defined inside and outside the class, a justified handling of the 11th transaction and a defended design.

ANSWER:

Q4. Specifying a Class with an Array Inside It — 6 Marks

4.1 C++ program: Savings account mini-statement

main.cpp

Run ▶

```cpp
#include <iostream>
#include <string>

class SavingsAccount {
private:
    int accountNumber;
    double balance;
    double transactions[10];
    int count;

    void record(double amount) {
        if (count == 10) {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];

            --count;
        }

        transactions[count++] = amount;
    }

public:
```

Output

```
Account: 1001
Balance: 5750
Recent transactions:
1000
-500
250
```

Expected output:

Account: 1001

Balance: 5750

Recent transactions:

1000

-500

250

4.2 Explanation

The class stores the account number, balance, transaction array and transaction count as private members. Private access prevents external code from directly changing the account balance or corrupting the transaction history.

Some member functions, such as the constructor and record(), are defined inside the class. The deposit(), withdraw() and miniStatement() functions are defined outside the class using the scope resolution operator SavingsAccount::.

When the 11th transaction arrives, the shifting approach removes the oldest transaction by moving each remaining element one position to the left.

4.3 Shifting versus a circular buffer

Feature	Array with shifting	Circular buffer
Adding a transaction when full	O(k)	O(1)
Storage	O(k)	O(k)
Implementation	Simple	Requires start/count indexing
Older transactions	Shifted out	Overwritten logically

Here, \(k\) is the maximum number of stored transactions. A std::deque can also support push_back() and pop_front() efficiently, generally in O(1) time per operation.

Design decision: For 50 transactions, a circular buffer is a good choice because the capacity can be changed to 50 without repeatedly shifting the entire history. It also provides predictable insertion time.

4.4 Alternative concept

Using a circular buffer, maintain a start index and a count. Insert the next transaction at (start + count) % capacity while the buffer is not full. When full, overwrite the oldest element and advance start modulo the capacity. This avoids the O(k) shifting operation.

Q5 – Static Members and Arrays of Objects 7 Marks

Bloom’s Level: K5 – Evaluate

Problem Context – College Bus Seat Tickets

The college bus has a fixed number of seats. Each BusTicket object must receive the next seat number, tickets beyond the capacity go to a waiting list, and the transport office must know how many seats have been issued without looking at any particular ticket.

Design Starting Point

You are given only the following principle:

Data that belongs to the whole class – not to any one object – should exist only once, however many objects are created.

Your Challenge

Convert this principle into a class design. Determine:

- Which members must be static, and where a static data member must be defined
- Why a static member function cannot use this or a non-static member
- How an array of BusTicket objects is created and initialised
- What happens to the count when tickets are created inside a loop or a function

Test Your Design

Create an array of tickets larger than the capacity and verify the seat numbers and the waiting list.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output. #include <iostream>

```
class Token {
   static int next;
int id;

public:
   Token() { id = ++next; }     int get()
const { return id; }     static int issued() {
return next - 100; }
```

```
};

int Token::next = 100;

int main()
{
   Token counter[3];     Token extra;     std::cout << counter[0].get() << ' '
<< counter[2].get() << ' ' << extra.get()              << ' ' << Token::issued()
<< '\n';
}
```

My predicted output (write before running):

Explain why the static counter starts at 100 and how many times the constructor ran.

Complete the Code

Complete TODO 1 and TODO 2 below using the static members, and execute the program.

```
#include <iostream>
#include <string>

class BusTicket {     static int issued;               // shared by all
BusTicket objects     static const int CAPACITY = 3;
   int seat;
std::string name;
```

```
public:     explicit BusTicket(const std::string& n) : seat(0),
name(n)
   {
      // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
      //   give this ticket the next seat number; otherwise leave seat as 0 (waiting
      //   list)

   }

   static int count()
   {
      // TODO 2: Return the number of seats issued so far

   }

   void print() const
   {
      std::cout << name << ": "
                << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                << '\n';
   }
};

int BusTicket::issued = 0;            // definition of the static data member

int main()
{
   BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
BusTicket("Chitra"), BusTicket("Dev")};     for (const
```

BusTicket& t : trip) t.print(); std::cout << "Issued: " <<

BusTicket::count() << '\n';

}

Expected output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

Create the tickets inside a function that is called twice and explain how the count behaves.

Alternative Concept and Complexity Analysis

If the seat counter were a global variable, or were kept in a separate TicketOffice object passed to every BusTicket, instead of a static data member, what would the solution be? Analyse the time (O(1) in all) and the space (one counter in all), and compare the encapsulation and the ability to run two buses at once.

Evidence Expected

A student-derived class design using static members, an array of objects, test evidence and a reasoned explanation of static members.

ANSWER:

Q5. Static Members and Arrays of Objects — 7 Marks

5.1 Predict the output

main.cpp Run ▶ Output

```cpp
#include <iostream>

class Token {
    static int next;
    int id;

public:
    Token() {
        id = ++next;
    }

    int get() const {
        return id;
    }

    static int issued() {
        return next - 100;
    }
};

int Token::next = 100;
```

```
101 103 104 4
```

Predicted output:

101 103 104 4

Explanation: The static counter starts at 100. The three objects in counter receive IDs 101, 102 and 103. The extra object receives ID 104. Therefore, the constructor runs four times, and issued() returns \(104-100=4\).

5.2 Complete the BusTicket code

main.cpp Run ▶ Output

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;

    int seat;
    std::string name;

public:
    explicit BusTicket(const std::string& n)
        : seat(0), name(n) {
        if (issued < CAPACITY) {
            seat = ++issued;
        }
    }

    static int count() {
        return issued;
    }
```

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Expected output:

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

5.3 Why static members are used

A static data member belongs to the class rather than an individual object. It exists once and is shared by every object of that class.

The issued variable is static because all tickets share the same seat counter. The count() function is static because it reads only class-level data and does not require a particular ticket object.

A static member function has no this pointer and cannot directly access non-static members, since those members belong to individual objects.

The static data member is defined outside the class:

int BusTicket::issued = 0;

If ticket objects are created inside a loop or a function, the shared counter continues increasing. Destroying the local objects does not reset the static counter.

5.4 Alternative concept and complexity

Global counter: A global variable can be accessed by unrelated code, weakening encapsulation.

TicketOffice object: A separate office object can own the counter and be passed to each ticket. This allows separate offices or buses to maintain independent counters.

Both approaches have O(1) counter-update time and O(1) counter storage per counter. A static counter is simple when one shared bus capacity is intended; separate office objects are preferable when multiple buses must issue seats independently.

Q6 – Friend Functions and Inline Functions 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Train Journey Planner

A journey planner stores departure and arrival times in a Time class whose hours and minutes are private. It must compute the minutes between two times and decide which time is earlier, and these small functions are called millions of times while planning connections.

Your Challenge

Design the comparison. Determine:

- Why a friend function is needed when the function works on two objects equally

- How a friend function differs from a member function in how it is called and what it can access
- Whether the small functions should be inline, and what inline actually asks the compiler to do
- What would be lost if the private members were made public instead

Correctness Check

Test with times on either side of noon and the same time twice, and verify by hand.

Growth Investigation

Measure the time for 10,000,000 calls with and without inline (compile with -O0 and with O2), and explain the result.

Deployment Decision

Decide whether the friend functions should stay or be replaced by member functions and getters, and defend your decision.

Complete the Code

Complete TODO 1 and TODO 2 below inside the friend functions, and execute the program.

```
#include <iostream>


class Time {
   int h, m;


public:
   Time(int hh, int mm) : h(hh), m(mm) {}      friend int
minutesBetween(const Time& a, const Time& b);     friend
bool earlier(const Time& a, const Time& b);
};


int minutesBetween(const Time& a, const Time& b)
{
```

```
    // TODO 1: Using the private members h and m, return the absolute difference between
//   the two times in minutes


}

bool earlier(const Time& a, const Time& b)
{
    // TODO 2: Return true if a is earlier in the day than b


}

int main()
{
    Time departure(9, 45), arrival(13, 10);     std::cout <<
minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output:

205 false

Test the same time twice and a pair of times on either side of noon.

Alternative Concept and Complexity Analysis

If the comparison were written as member functions (a.minutesTo(b), a.isEarlier(b)) or with public getters instead of friend functions, what would the solution be? Analyse the time per call (the same after inlining, O(1)) and measure it at -O0 and -O2, and compare the encapsulation of the three designs.

Evidence Expected

A justified friend-function design, correctness tests, measured inline timings at two optimisation levels and an evidence-based decision.

ANSWER:

Q6. Friend Functions and Inline Functions — 8 Marks

6.1 Complete the code



Expected output:

205 false

6.2 Explanation

The time 9:45 is equal to $9 \times 60 + 45 = 585$ minutes after midnight. The time 13:10 is equal to $13 \times 60 + 10 = 790$ minutes after midnight.

The difference is $790 - 585 = 205$ minutes. The expression earlier(arrival, departure) returns false because 13:10 is later than 9:45.

6.3 Correctness tests

Add the following tests to verify identical times and times on either side of noon:

main.cpp

```cpp
Time noon(12, 0);
Time beforeNoon(11, 59);
Time sameTime(12, 0);

std::cout << minutesBetween(noon, beforeNoon) << '\n';
std::cout << std::boolalpha
          << earlier(beforeNoon, noon) << '\n';
std::cout << minutesBetween(noon, sameTime) << '\n';
std::cout << earlier(noon, sameTime) << '\n';
```

These tests confirm that the functions handle adjacent minutes, identical times and chronological comparison correctly. The implementation assumes both times are valid and belong to the same day.

6.4 Friend functions versus member functions

A friend function is not a member of the class, but it can access private members because the class explicitly grants friendship. It is called like an ordinary function:

minutesBetween(departure, arrival);

A member function is called through an object:

departure.minutesBetween(arrival);

A friend function is appropriate when a function compares two objects symmetrically. However, public getters or member functions can preserve stronger encapsulation by keeping direct private-member access inside the class.

6.5 Inline functions and performance

An inline function allows the compiler to consider replacing a function call with the function body. The inline keyword does not guarantee that this substitution will happen. Compilers can inline functions automatically when appropriate.

A possible member-function alternative is:

int minutesTo(const Time& other) const;

bool isEarlier(const Time& other) const;

These methods can perform the same calculations with O(1) time complexity and O(1) additional space.

Performance experiment: Compile equivalent implementations using -O0 and -O2, then measure 10,000,000 calls. Use std::chrono and ensure the results are used so the compiler cannot eliminate the computation. Repeat the measurements and compare the results.

g++ -std=c++17 -O0 timing.cpp -o timing_O0

g++ -std=c++17 -O2 timing.cpp -o timing_O2

Do not claim a particular timing improvement without running the experiment. Optimization level, compiler version, hardware and measurement overhead can affect the result.

6.6 Deployment decision

Friend functions can remain when symmetrical comparison and direct access to private members provide a clear benefit. If the class should hide its representation more strictly, member functions or public getters are alternatives. Since the operations are small and have constant complexity, the final choice should prioritize correctness, maintainability and measured performance rather than assuming inline is always faster.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE program that shows a significant limitation as the data grows or the requirement changes.

You are not permitted simply to replace your code with a library facility and claim it is better. You must understand the problem first and then develop an appropriate redesign.

A. Diagnose the Limitation – 2 Marks

Using your own evidence, identify where the design fails or slows down and which characteristic of the original design causes it.

Evidence Expected

A precise diagnosis supported by evidence from your original program.

B. Define the Design Objective – 2 Marks

State a measurable objective for the redesign.

Evidence Expected

A clearly stated objective derived from the identified limitation.

C. Design the Revised Solution – 4 Marks

Explain what you have changed in the class or function design, which C++ feature you use, whether extra memory is needed, and any new trade-off. Provide a class diagram or pseudocode.

Evidence Expected

A coherent redesign directly addressing the diagnosed limitation.

D. Predict Before Implementing – 1 Mark

Predict the improvement and the measure in which you expect it.

Evidence Expected

A clear, testable prediction.

E. Implement and Validate the Redesign – 2 Marks

Compare Original → Redesigned using the same tests and measurements.

Evidence Expected

Comparable evidence for both versions.

F. Defend the Final Design – 2 Marks

Evaluate whether the objective was achieved, new costs, and where the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly explain why.

Evidence Expected

A critical, evidence-based evaluation and a defensible conclusion.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q7
- C++ source code for all programs
- Predictions, outputs, timings and compiler messages used as evidence

Assessment Verification

Students may be asked to explain any part of their predictions, code, tests or results during a brief verification discussion/viva.